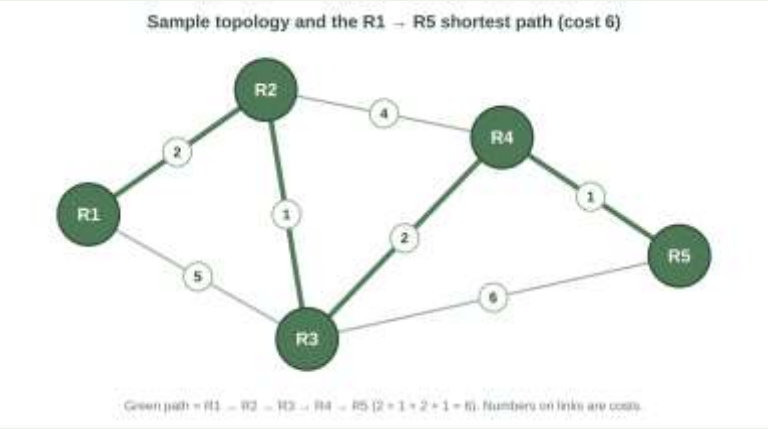
Interactive Network Routing Simulator

RouteLab: Dijkstra and Distance Vector with congestion and link failure

Presented by: Nikhila Haasini Gadamsetty, Nitu S U
Course: Computer Networks Lab

Why this project?

- Routers decide the path of every packet
- Textbooks show the result, not the process
- Hard to see what happens when a link gets congested or fails
- Goal: a simulator where you can build a network and try it



Sample network used in the demo (R1 → R5, cost 6)

Objectives

Build networks

Add and delete routers, links, costs, save and load

Dijkstra

Fast shortest path with a trace

Distance Vector

Message exchange, split horizon, count-to-infinity

Congestion

Link load slider changes the cost

Failures

Failed links, partitions, unreachable

Verification

Automated tests

Technology used

Next.js 15

pages and routing

React 19

UI components

React Flow

draggable network graph

Zustand

one shared store

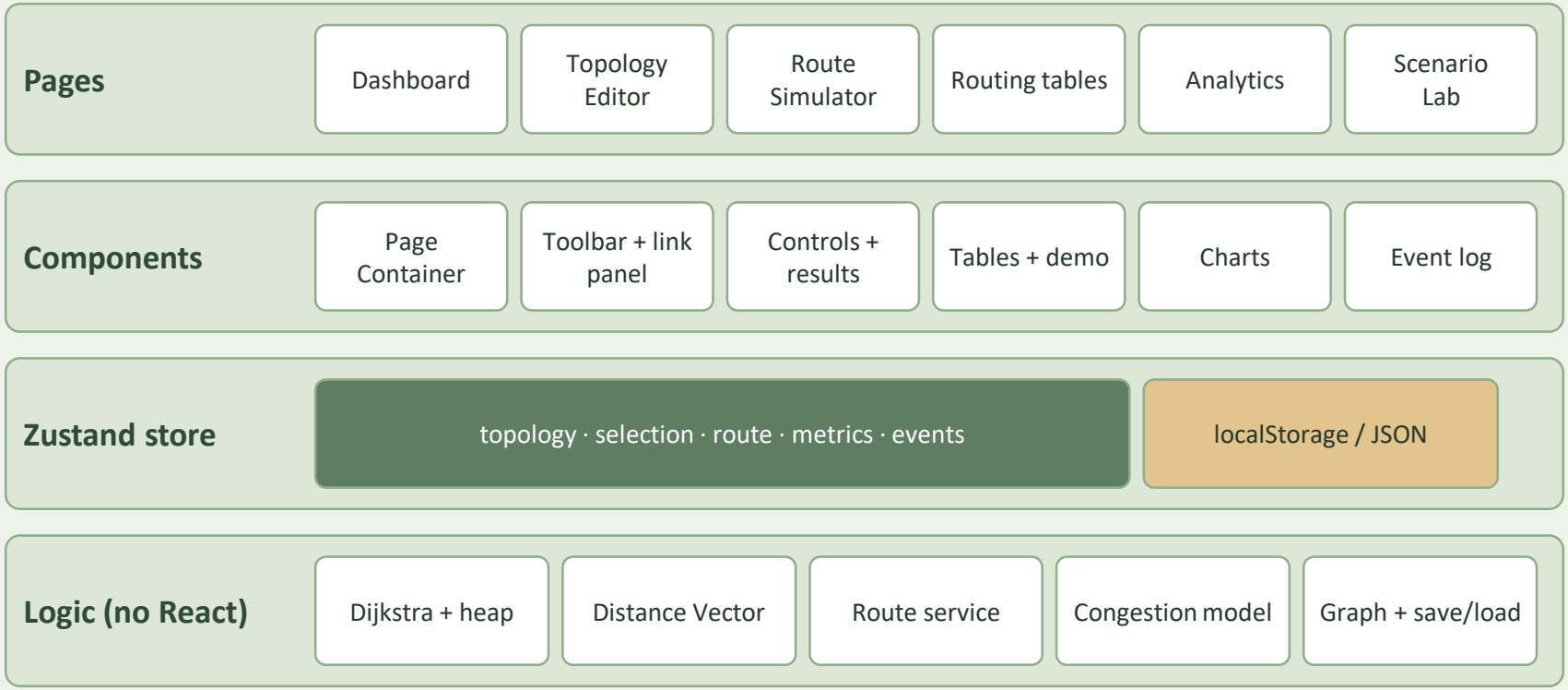
TypeScript

typed algorithms

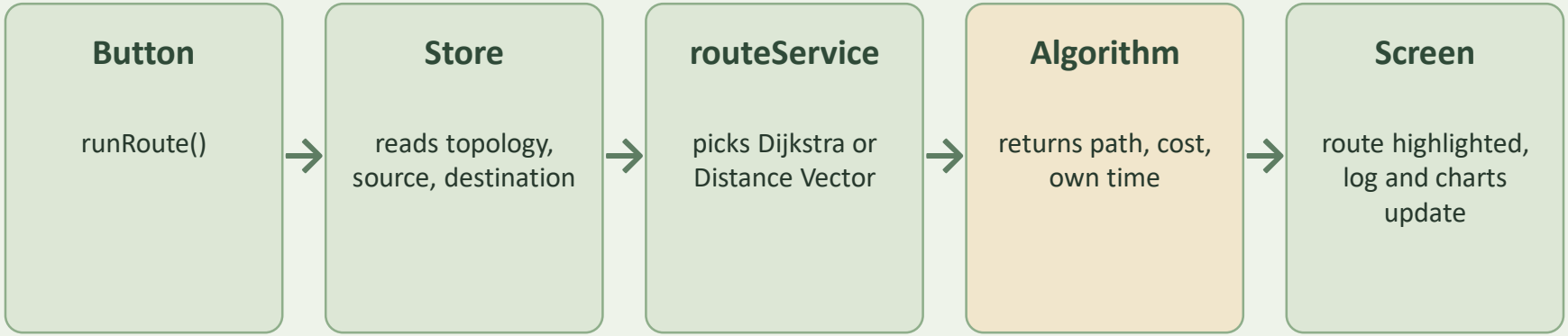
Vitest

automated tests

System architecture



What happens when you click “Run”



After any change (cost, load, failed link, deleted router) the store recomputes the route on screen automatically, so the highlighted path is never out of date.

Dijkstra’s algorithm

- Needs the whole map
- Pick the closest unsettled router, update its neighbours
- Follow ‘previous’ links back to get the path
- Result for R1 → R5: cost 6

Settled	R1	R2	R3	R4	R5
start	0	∞	∞	∞	∞
R1	0	2	5	∞	∞
R2	0	2	3	6	∞
R3	0	2	3	5	9
R4	0	2	3	5	6

Distances from R1 after each router is settled

Making Dijkstra faster

	Before	After
Finding next router	scan all routers	binary heap
Finding neighbours	scan all links	adjacency list
Complexity	about $O(V^2 + V \cdot E)$	$O((V + E) \log V)$

≈ 12×

faster on 2,000 routers
(104 ms → 8.5 ms)

Same results as the old version, and checked against a reference algorithm on a 300-router random network.

Distance Vector routing

- Every round each router sends its table to its neighbours
- cost(x,y) = min over neighbours n of link(x,n) + n's advertised cost to y
- Route is built by following next hops, router by router
- Sample network: 5 rounds, 70 messages

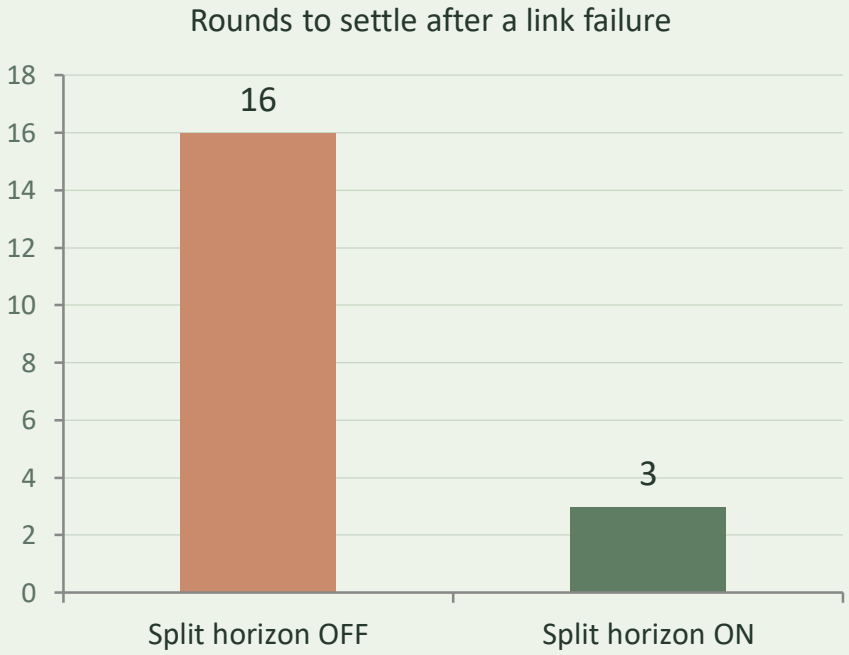
Round	R2	R3	R4	R5
1	2	5	∞	∞
2	2	3	6	11
3	2	3	5	7
4	2	3	5	6
5	2	3	5	6

R1's cost to each router after each round (no change in round 5 = converged)

Dijkstra vs Distance Vector

	Dijkstra (link state)	Distance Vector
Router knows	whole topology	own links + neighbours' tables
Real protocol	OSPF	RIP
Convergence	immediate once map is known	takes rounds of messages
Known problem	needs the full map	count-to-infinity, loops
In my project	heap-based, with trace	message exchange, split horizon

Count-to-infinity and split horizon



- After a failure, routers keep raising each other's cost estimate
- Metric 16 means 'unreachable' and stops it (as in RIP)
- Split horizon: never advertise a route back to where it came from
- Poison reverse: advertise it back as unreachable
- Live demo on the Routing tables page

Congestion and link failure

cost = base × (1 + load / 100)

load slider 0–100 %, set by the user

R1 → R5	Path	Cost
Normal	R1 R2 R3 R4 R5	6
L5 and L3 at 100 % load	R1 R2 R4 R5	7
L5 failed	R1 R2 R4 R5	7
L6, L7 failed	no route	∞

- Route on screen is recalculated automatically after every change
- Event log explains the failure recovery
- Both algorithms handle unreachable routers and partitions

What was fixed and added

Real DV path

route built from next hops, own timing

No stale route

auto-recalculated after changes

Router positions

dragging is remembered

Delete router

its links are removed too

Save / load

browser storage or JSON file

Unused code

all six functions connected

Testing

93

automated tests
in 12 files

Area	Tests
Store actions	21
Distance Vector	16
Dijkstra + heap	9
Graph, save/load, metrics	17
Packets, partitions, tables, scenarios, recovery	23
Congestion	7

Not covered by automated tests: dragging, sliders and file import in the browser (checked by hand).

Live demo plan

1. Open the sample network, drag a router
2. Run R1 → R5 with Dijkstra (cost 6)
3. Switch to Distance Vector and run again
4. Raise the load on R3–R4: route updates by itself
5. Disable R3–R4: read the recovery message
6. Routing tables: DV rounds and count-to-infinity demo
7. Network partition scenario: unreachable
8. Save the topology and load it again

Limitations and future work

Limitations

- DV runs in lock-step rounds, no timers (not real RIP)
- DV treats cost ≥ 16 as unreachable
- Load is set by the user, not measured
- Packet journey is step-based
- No automated browser (UI) tests

Future work

- Asynchronous DV with timers and message loss
- Link-state protocol with flooding (OSPF style)
- Traffic-based congestion and queues
- Directed links, undo / redo
- End-to-end browser tests

Conclusion

- RouteLab shows how routes are computed and how they react to congestion and failures
- Dijkstra (heap) and Distance Vector (messages, split horizon) give the same costs, checked by tests
- The route on screen always comes from the selected algorithm and is never stale
- 93 automated tests; limitations are documented
- This is an educational simulation, not a live simulation

Thank you.